Data Science · Day 12 of 30 · about 60 minutes

Cleaning messy data

🎯 Today's goal

By the end of this hour you will:

- **spot** problems in a table: empty cells, duplicate rows, wrong types, odd values
- remove or fill empty cells with `dropna` and `fillna`
- tidy text (extra spaces, capital letters, different spellings) and fix column names
- turn text into numbers with `pd.to_numeric` and into dates with `pd.to_datetime`
- find impossible values (like an age of 250) and fix them with `.loc`

🤔 Why this matters

Yesterday's café data was perfectly tidy. Real data almost never is. People type *"cairo"*, *"Cairo "* and *"CAIRO"* for the same city, forms get sent twice, someone writes *"unknown"* where a number should be, and a slip of the finger turns an age of 25 into 250.

Cleaning data is like washing vegetables before cooking: not exciting, but skip it and the whole meal is spoiled. Data scientists often say they spend **most of their time** cleaning. A chart or a machine-learning model built on messy data gives wrong answers without any warning, so this is one of the most useful skills in the whole course.

🧾 Today's data: a messy customer list

A small online shop exported its customer list: an ID, the name, age, city, the date they signed up and how much they spend per month (in dollars). The data is **made up**, and we planted these problems in it on purpose:

- column names with capital letters and spaces
- **empty cells** (missing ages and spending)
- 3 **duplicate rows** (the same customer twice)
- city names written in different ways (*cairo*, *Cairo*, *CAIRO*, *Alex*...)
- numbers stored as **text**, including the words *unknown* and *ninety*
- **impossible values**: an age of 250 and negative spending
- dates stored as plain text

```
import io
import numpy as np
import pandas as pd

# Made-up, deliberately messy customer list
csv_text = """Customer ID,Name,Age,City,Signup Date,Monthly Spend
1,Sara,28,Cairo,2026-01-05,120.5
2,Omar,35,cairo,2026-01-12,80
3,Lina,,Alexandria,2026-01-20,95
4,Sami,41,GIZA,2026-02-03,unknown
5,Rana,250,Cairo,2026-02-10,60
6,Youssef,23,Alex,2026-02-14,-15
7,Mona,31, giza,2026-02-20,150
8,Karim,,Cairo,2026-03-03,70.25
9,Huda,45,Alexandria ,2026-03-05,110
10,Tarek,38,Giza,2026-03-09,
11,Nour,26,cairo ,2026-03-15,85.5
12,Adel,52,Alex,2026-03-22,200
13,Salma,29,Giza,2026-04-01,ninety
14,Fadi,33,Cairo,2026-04-04,65
15,Dina,,Alexandria,2026-04-10,130
16,Hany,47,CAIRO,2026-04-18,95
17,Laila,24,Giza,2026-04-25,-40
18,Ziad,36,Alexandria,2026-05-02,75
19,Reem,30, Cairo,2026-05-09,140
20,Bassem,44,Giza,2026-05-16,55
21,Maya,27,Alex,2026-05-23,90
22,Nabil,39,Cairo,2026-05-30,105
2,Omar,35,cairo,2026-01-12,80
9,Huda,45,Alexandria ,2026-03-05,110
14,Fadi,33,Cairo,2026-04-04,65
"""

customers = pd.read_csv(io.StringIO(csv_text))
print(customers.shape)
customers.head(8)
```

```
(25, 6)
```

	Customer ID	Name	Age	City	Signup Date	Monthly Spend
0	1	Sara	28.0	Cairo	2026-01-05	120.5
1	2	Omar	35.0	cairo	2026-01-12	80
2	3	Lina	NaN	Alexandria	2026-01-20	95
3	4	Sami	41.0	GIZA	2026-02-03	unknown
4	5	Rana	250.0	Cairo	2026-02-10	60
5	6	Youssef	23.0	Alex	2026-02-14	-15
6	7	Mona	31.0	giza	2026-02-20	150
7	8	Karim	NaN	Cairo	2026-03-03	70.25

See the `NaN` in the *Age* column? That's how pandas shows an **empty cell**. NaN stands for "Not a Number"; you'll also hear *missing value*.

🔎 Step 1: spot the problems

Before fixing anything, look. You met `info()` on Day 10. Read it like a doctor's check-up: the *Non-Null Count* says how many cells are **filled** in each column (25 means none are missing), and the *Dtype* says what type each column has:

```
customers.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 25 entries, 0 to 24
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Customer ID    25 non-null     int64
 1   Name           25 non-null     str
 2   Age            22 non-null     float64
 3   City           25 non-null     str
 4   Signup Date    25 non-null     str
 5   Monthly Spend  24 non-null     str
dtypes: float64(1), int64(1), str(4)
memory usage: 872.0 bytes
```

Two warning signs: *Age* and *Monthly Spend* have fewer than 25 filled cells, and *Monthly Spend* is `str` (text), although it should hold numbers. A faster way to count the empty cells is `isna()` ("is it NaN?"), which gives True for every empty cell. Adding up True values counts them, because True counts as 1:

```
print(customers.isna().sum())          # empty cells per column
print("Total:", customers.isna().sum().sum())
```

```
Customer ID      0
Name             0
Age              3
City             0
Signup Date      0
Monthly Spend    1
dtype: int64
Total: 4
```

`duplicated()` gives True for every row that is an exact copy of an earlier row. With `keep=False` it marks *all* copies, so you can see the pairs:

```
print("Duplicate rows:", customers.duplicated().sum())
customers[customers.duplicated(keep=False)]
```

```
Duplicate rows: 3
```

	Customer ID	Name	Age	City	Signup Date	Monthly Spend
1	2	Omar	35.0	cairo	2026-01-12	80
8	9	Huda	45.0	Alexandria	2026-03-05	110
13	14	Fadi	33.0	Cairo	2026-04-04	65
22	2	Omar	35.0	cairo	2026-01-12	80
23	9	Huda	45.0	Alexandria	2026-03-05	110
24	14	Fadi	33.0	Cairo	2026-04-04	65

🧹 Step 2: duplicates and column names

`drop_duplicates()` keeps the first copy of each row and drops the rest. We store the result in a new variable, `clean`, so the original stays untouched for comparison. Then `rename` gives the

columns easy names: a dictionary of `{"old name": "new name"}`. Short snake_case names (Day 2) are much nicer to type than `customers["Monthly Spend"]`:

```
clean = customers.drop_duplicates()
print("Rows before:", len(customers), " after:", len(clean))

clean = clean.rename(columns={
    "Customer ID": "customer_id",
    "Name": "name",
    "Age": "age",
    "City": "city",
    "Signup Date": "signup_date",
    "Monthly Spend": "spend",
})
print(list(clean.columns))
```

```
Rows before: 25  after: 22
['customer_id', 'name', 'age', 'city', 'signup_date', 'spend']
```

🔤 Step 3: tidy the text

How many cities are there? Count them with `value_counts()`:

```
clean["city"].value_counts()
```

```
city
Cairo          5
Giza           4
Alexandria     3
Alex           3
cairo          1
GIZA           1
 giza          1
Alexandria     1
cairo          1
CAIRO          1
 Cairo         1
Name: count, dtype: int64
```

Eleven "cities", but really only three! On Day 2 you tidied one piece of text with `.strip()` and `.title()`. On a pandas column you write `.str` first, and the tool is applied to **every** cell at once. Then `replace` swaps a nickname for the full name, using a dictionary again:

```
clean["city"] = clean["city"].str.strip().str.title()   # " cairo " -> "Cairo"
clean["city"] = clean["city"].replace({"Alex": "Alexandria"})

clean["city"].value_counts()
```

```
city
Cairo         9
Alexandria    7
Giza          6
Name: count, dtype: int64
```

Three cities, as it should be. (`.str.lower()` and `.str.upper()` work too.)

🔢 Step 4: numbers stored as text

The `spend` column is text, because a few cells contain words. Text can't be averaged, so this fails:

💡 This example has a mistake on purpose. Run it and read the error.

```
clean["spend"].mean()
```

⚠️ TypeError

You combined things that don't fit together, like text + a number.

```
Traceback (most recent call last):
  line 1, in <module>
    clean["spend"].mean()
    ~~~~~~~~~~~~~~~~~~^^
TypeError: Cannot perform reduction 'mean' with string dtype
```

"Cannot perform reduction 'mean' with string dtype" means: you can't average text. `astype(float)` would fail too, on the word *unknown*. `pd.to_numeric(..., errors="coerce")` converts what it can and turns everything else into NaN. (*Coerce* means "force": "if you can't convert it, make it empty".)

```
clean["spend"] = pd.to_numeric(clean["spend"], errors="coerce")

print(clean["spend"].dtype)
print("Empty spend cells now:", clean["spend"].isna().sum())
print(f"Average spend: {clean['spend'].mean():.2f}")
```

```
float64
Empty spend cells now: 3
Average spend: 87.96
```

Now it works. *unknown*, *ninety* and the empty cell became NaN, and `mean()` simply skips NaN. But that average is still wrong. Why? Read on.

🚩 Step 5: impossible values

Some values are filled in but can't be true: nobody is 250 years old, and nobody spends a negative amount. Find them with a filter (Day 11). The `|` means "or":

```
clean[(clean["age"] > 120) | (clean["spend"] < 0)]
```

	customer_id	name	age	city	signup_date	spend
4	5	Rana	250.0	Cairo	2026-02-10	60.0
5	6	Youssef	23.0	Alexandria	2026-02-14	-15.0
16	17	Laila	24.0	Giza	2026-04-25	-40.0

We don't know the true values, so we mark them as missing with `np.nan` (NumPy's empty value). `.loc[rows, column]` from Day 10 changes only the cells where the condition is True. Remember: in pandas 3 you must use `.loc` to change part of a table; `clean["age"][4] = ...` does nothing.

```
clean.loc[clean["age"] > 120, "age"] = np.nan
clean.loc[clean["spend"] < 0, "spend"] = np.nan

print(clean.isna().sum())
```

```
customer_id    0
name           0
age            4
city           0
signup_date    0
spend          5
dtype: int64
```

🕳️ Step 6: drop or fill the empty cells?

There are two choices for an empty cell, and both are fine in the right place:

Method	What it does	Good when…
`dropna()`	removes rows with an empty cell	only a few rows are affected, or the value can't be guessed
`fillna(value)`	writes a value into the empty cells	a typical value is a fair guess (the median, the mean, 0, "Unknown")

For ages we fill in the **median** (the middle value), which isn't pulled up or down by unusual ages. For spending, a guess would distort exactly the number we care about, so we drop those rows. `subset=["spend"]` means "only look at the spend column":

```
median_age = clean["age"].median()
print("Median age:", median_age)
clean["age"] = clean["age"].fillna(median_age)

print("Rows before dropping:", len(clean))
clean = clean.dropna(subset=["spend"])
print("Rows after dropping:", len(clean))
print(f"Average spend (clean): {clean['spend'].mean():.2f}")
```

```
Median age: 34.0
Rows before dropping: 22
Rows after dropping: 17
Average spend (clean): 101.54
```

Be honest about it. Every fill or drop changes your results a little. Write down what you did ("4 missing ages filled with the median, 5 rows without a valid spend removed") so anyone reading your analysis knows.

📅 Step 7: real dates

`signup_date` is text that only *looks* like a date. `pd.to_datetime` turns it into a real date. Then `.dt` (like `.str`, but for dates) gives you parts of it, such as `.dt.month` or `.dt.year`:

```
clean["signup_date"] = pd.to_datetime(clean["signup_date"])
clean["month"] = clean["signup_date"].dt.month

print(clean["signup_date"].dtype)
print("Sign-ups per month:")
print(clean["month"].value_counts().sort_index())
```

```
datetime64[us]
Sign-ups per month:
month
1    3
2    2
3    4
4    3
5    5
Name: count, dtype: int64
```

🏁 The result

Run `info()` again and compare it with Step 1: no empty cells, sensible types and short column names. `reset_index(drop=True)` renumbers the rows 0, 1, 2… (the gaps came from the rows we removed):

```
clean = clean.reset_index(drop=True)
clean.info()
clean.head()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 17 entries, 0 to 16
Data columns (total 7 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   customer_id  17 non-null     int64
 1   name         17 non-null     str
 2   age          17 non-null     float64
 3   city         17 non-null     str
 4   signup_date  17 non-null     datetime64[us]
 5   spend        17 non-null     float64
 6   month        17 non-null     int32
dtypes: datetime64[us](1), float64(2), int32(1), int64(1), str(2)
memory usage: 820.0 bytes
```

	customer_id	name	age	city	signup_date	spend	month
0	1	Sara	28.0	Cairo	2026-01-05	120.5	1
1	2	Omar	35.0	Cairo	2026-01-12	80.0	1
2	3	Lina	34.0	Alexandria	2026-01-20	95.0	1
3	5	Rana	34.0	Cairo	2026-02-10	60.0	2
4	7	Mona	31.0	Giza	2026-02-20	150.0	2

🧭 A cleaning checklist

1. **Look:** `shape`, `head()`, `info()`, `isna().sum()`, `duplicated().sum()`.
2. **Duplicates:** `drop_duplicates()`.
3. **Names:** `rename(columns={...})` into short snake_case.
4. **Text:** `.str.strip()`, `.str.title()`, `replace({...})`; check with `value_counts()`.
5. **Types:** `pd.to_numeric(..., errors="coerce")`, `pd.to_datetime(...)`.
6. **Impossible values:** find with a filter, set to `np.nan` with `.loc`.
7. **Empty cells:** `fillna` or `dropna`, and write down which.
8. **Check again:** `info()` and `isna().sum()`.

⚠️ Common mistakes

- Forgetting to store the result: `clean.dropna()` alone changes nothing. Write `clean = clean.dropna()`.
- Forgetting `.str`: `clean["city"].strip()` gives an `AttributeError`; write `clean["city"].str.strip()`.
- Changing cells without `.loc`: `clean["age"][4] = 30` does nothing in pandas 3.
- Dropping too much: `dropna()` without `subset` removes a row if *any* column is empty.
- Filling everything with 0: a missing age is not age 0. Choose a value that makes sense.

✅ Recap

- Find problems with `info()`, `isna().sum()`, `duplicated().sum()` and `value_counts()`.
- `drop_duplicates()` removes copies; `rename(columns={...})` fixes names.
- `.str.strip()`, `.str.lower()`, `.str.title()` and `replace({...})` tidy text.
- `pd.to_numeric(col, errors="coerce")` makes numbers (bad values become NaN); `pd.to_datetime` makes dates, and `.dt.month` gives the month.
- Impossible values: `df.loc[condition, "col"] = np.nan`.
- Empty cells: `fillna(value)` or `dropna(subset=[...])`.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Count the empty cells

A weather station lost some readings. Store the number of empty cells **per column** in `missing_counts`, and the **total** number of empty cells in `total_missing`.

```
import io
import pandas as pd

# Made-up weather readings with gaps
csv_text = """day,city,temp,rain_mm
Mon,Cairo,24.5,0
Tue,Cairo,,0
Wed,Cairo,26.1,
Thu,Cairo,,2.5
Fri,Cairo,25.0,0
Sat,,23.8,1.0
"""
weather = pd.read_csv(io.StringIO(csv_text))

# Create missing_counts and total_missing below
```

Hints:

1. `weather.isna()` gives True for every empty cell.
2. `.sum()` after it counts the Trues in each column.
3. `total_missing = missing_counts.sum()`

2. Fill the gaps

Fill the missing temperatures with the **median** temperature. Store the median in `temp_median`, and update the `temp` column of `weather` so it has no empty cells. Leave the other columns as they are.

```
import io
import pandas as pd

# Made-up weather readings with gaps
csv_text = """day,city,temp,rain_mm
Mon,Cairo,24.5,0
Tue,Cairo,,0
Wed,Cairo,26.1,
Thu,Cairo,,2.5
Fri,Cairo,25.0,0
Sat,,23.8,1.0
"""
weather = pd.read_csv(io.StringIO(csv_text))

# Create temp_median, then fill the temp column
```

Hints:

1. `temp_median = weather["temp"].median()` (it skips the empty cells).
2. `weather["temp"].fillna(temp_median)` gives a filled copy of the column.
3. Store it back: `weather["temp"] = weather["temp"].fillna(temp_median)`

3. Orders sent twice

Some orders were saved twice. Store the **number of duplicate rows** in `n_duplicates`, and a table without them in `unique_orders`.

```
import io
import pandas as pd

# Made-up orders; some were saved twice
csv_text = """order_id,product,qty
101,Coffee,2
102,Tea,1
103,Cake,3
102,Tea,1
104,Juice,2
105,Coffee,1
103,Cake,3
106,Tea,4
"""
orders = pd.read_csv(io.StringIO(csv_text))

# Create n_duplicates and unique_orders below
```

Hints:

1. `orders.duplicated()` gives True for each repeated row; `.sum()` counts them.
2. `unique_orders = orders.drop_duplicates()`

4. Prices stored as text

The `price` column arrived as text, with a few words mixed in. Convert it to numbers with `pd.to_numeric` (words should become empty cells) and store it back in `products["price"]`. Then store the average price in `average_price`.

```
import pandas as pd

# Made-up product list; the prices came in as text
products = pd.DataFrame({
    "product": ["Pen", "Notebook", "Sticker", "Ruler", "Eraser", "Folder"],
    "price": ["2.50", "3.10", "free", "4.00", "?", "1.75"],
})

# Convert the price column, then create average_price
```

Hints:

1. `errors="coerce"` turns *free* and *?* into NaN instead of raising an error.
2. `products["price"] = pd.to_numeric(products["price"], errors="coerce")`
3. `average_price = products["price"].mean()` skips the empty cells.

5. Challenge: clean it all

Clean this messy list of exam scores into a variable called `clean`:

1. Rename the columns to `student`, `school` and `score`.
2. Remove duplicate rows.
3. Tidy the school names so they are exactly `North` or `South`.
4. Convert `score` to numbers (words become empty cells).
5. Keep only rows with a real score between 0 and 100 (this also removes the empty ones).

Finally, store the **average score per school** in `school_means` (a Series, using `groupby` from Day 11).

```
import io
import pandas as pd

# Made-up, messy exam results
csv_text = """Student,School,Score
Ali, north ,78
Bana,North,85
Ali, north ,78
Cyrus,SOUTH,abc
Dalia,south,105
Emad,South,64
Farah,north,
Ghada,South,91
Hadi,NORTH,-5
Iman,south,72
"""
scores = pd.read_csv(io.StringIO(csv_text))

# Build clean step by step, then school_means
```

Hints:

1. Start with `clean = scores.rename(columns={"Student": "student", ...})`, then `clean = clean.drop_duplicates()`.
2. Schools: `clean["school"] = clean["school"].str.strip().str.title()`. Scores: `pd.to_numeric(..., errors="coerce")`.
3. `clean = clean[clean["score"].between(0, 100)]` keeps only real scores (NaN counts as "not between"). Then `school_means = clean.groupby("school")["score"].mean()`

🔜 Tomorrow

Charts with Matplotlib: a table of numbers is hard to read, but a picture shows a trend in a second. Tomorrow you'll draw line charts, bar charts, histograms and scatter plots, give them titles and labels, and put two charts side by side.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Count the empty cells

```
import io
import pandas as pd

# Made-up weather readings with gaps
csv_text = """day,city,temp,rain_mm
Mon,Cairo,24.5,0
Tue,Cairo,,0
Wed,Cairo,26.1,
Thu,Cairo,,2.5
Fri,Cairo,25.0,0
Sat,,23.8,1.0
"""
weather = pd.read_csv(io.StringIO(csv_text))

missing_counts = weather.isna().sum()
total_missing = missing_counts.sum()
print(missing_counts)
print("Total:", total_missing)
```

2. Fill the gaps

```
import io
import pandas as pd

# Made-up weather readings with gaps
csv_text = """day,city,temp,rain_mm
Mon,Cairo,24.5,0
Tue,Cairo,,0
Wed,Cairo,26.1,
Thu,Cairo,,2.5
Fri,Cairo,25.0,0
Sat,,23.8,1.0
"""
weather = pd.read_csv(io.StringIO(csv_text))

temp_median = weather["temp"].median()
weather["temp"] = weather["temp"].fillna(temp_median)
print(weather)
```

3. Orders sent twice

```
import io
import pandas as pd

# Made-up orders; some were saved twice
csv_text = """order_id,product,qty
101,Coffee,2
102,Tea,1
103,Cake,3
102,Tea,1
104,Juice,2
105,Coffee,1
103,Cake,3
106,Tea,4
"""
orders = pd.read_csv(io.StringIO(csv_text))

n_duplicates = orders.duplicated().sum()
unique_orders = orders.drop_duplicates()
print("Duplicates:", n_duplicates)
print(unique_orders)
```

4. Prices stored as text

```
import pandas as pd

# Made-up product list; the prices came in as text
products = pd.DataFrame({
    "product": ["Pen", "Notebook", "Sticker", "Ruler", "Eraser", "Folder"],
    "price": ["2.50", "3.10", "free", "4.00", "?", "1.75"],
})

products["price"] = pd.to_numeric(products["price"], errors="coerce")
average_price = products["price"].mean()
print(products)
print(f"Average price: {average_price:.2f}")
```

5. Challenge: clean it all

```
import io
import pandas as pd

# Made-up, messy exam results
csv_text = """Student,School,Score
Ali, north ,78
Bana,North,85
Ali, north ,78
Cyrus,SOUTH,abc
Dalia,south,105
Emad,South,64
Farah,north,
Ghada,South,91
Hadi,NORTH,-5
Iman,south,72
"""
scores = pd.read_csv(io.StringIO(csv_text))

clean = scores.rename(columns={"Student": "student", "School": "school",
                               "Score": "score"})
clean = clean.drop_duplicates()
clean["school"] = clean["school"].str.strip().str.title()
clean["score"] = pd.to_numeric(clean["score"], errors="coerce")
clean = clean[clean["score"].between(0, 100)]

school_means = clean.groupby("school")["score"].mean()
print(clean)
print(school_means)
```